🌐 Module 06: API and Java Web Token Penetration Testing

```
# Exercise 10: Bruteforcing Weak Secret Keys in JSON Web Tokens (JWT) Implementations
```

Scenario

JSON Web Tokens (JWT) rely on cryptographic signatures to guarantee payload integrity and authenticity. When HMAC algorithms (e.g., `HS256`) are configured with weak, predictable, or dictionary-based secret keys (such as `access` or `secret`), unauthorized attackers can intercept the token, brute-force the signing key offline using dictionary attacks, and sign arbitrary forged claims.

In **DVWS-Node**, user sessions issue signed JWTs upon authentication (`test:test`). The session authorization token contains privilege claims (`["user:user"]`), preventing access to `/admin`. By extracting the Bearer token from Burp Suite Proxy history, offline brute-forcing via `jwt_tool` against SecLists wordlists (**10k-most-common.txt**) uncovers the signing key (**access**). Modifying payload claims to `["user:admin"]` and re-signing the token with the recovered secret enables authentication bypass and privilege escalation to the administrative control area.

In this exercise, we launch `dvws-node` on Ubuntu, capture user Bearer tokens in Burp Suite on Parrot Security, brute-force weak HMAC secret keys using `jwt_tool` (**access**), forge an admin JWT using `jwt.io`, and replace session tokens to gain full administrative access.

Objective

The objective of this lab is to help students learn how to:
- Intercept HTTP Authorization Bearer tokens in Burp Suite Proxy HTTP history
- Perform offline JWT HMAC secret key brute-force attacks using `jwt_tool` and dictionary wordlists
- Identify weak JWT HMAC signing secrets (**access**)
- Modify JWT payload claim structures to escalate user privileges to `["user:admin"]`
- Re-sign forged tokens using `jwt.io` or `jwt_tool` and substitute Authorization headers to bypass access controls

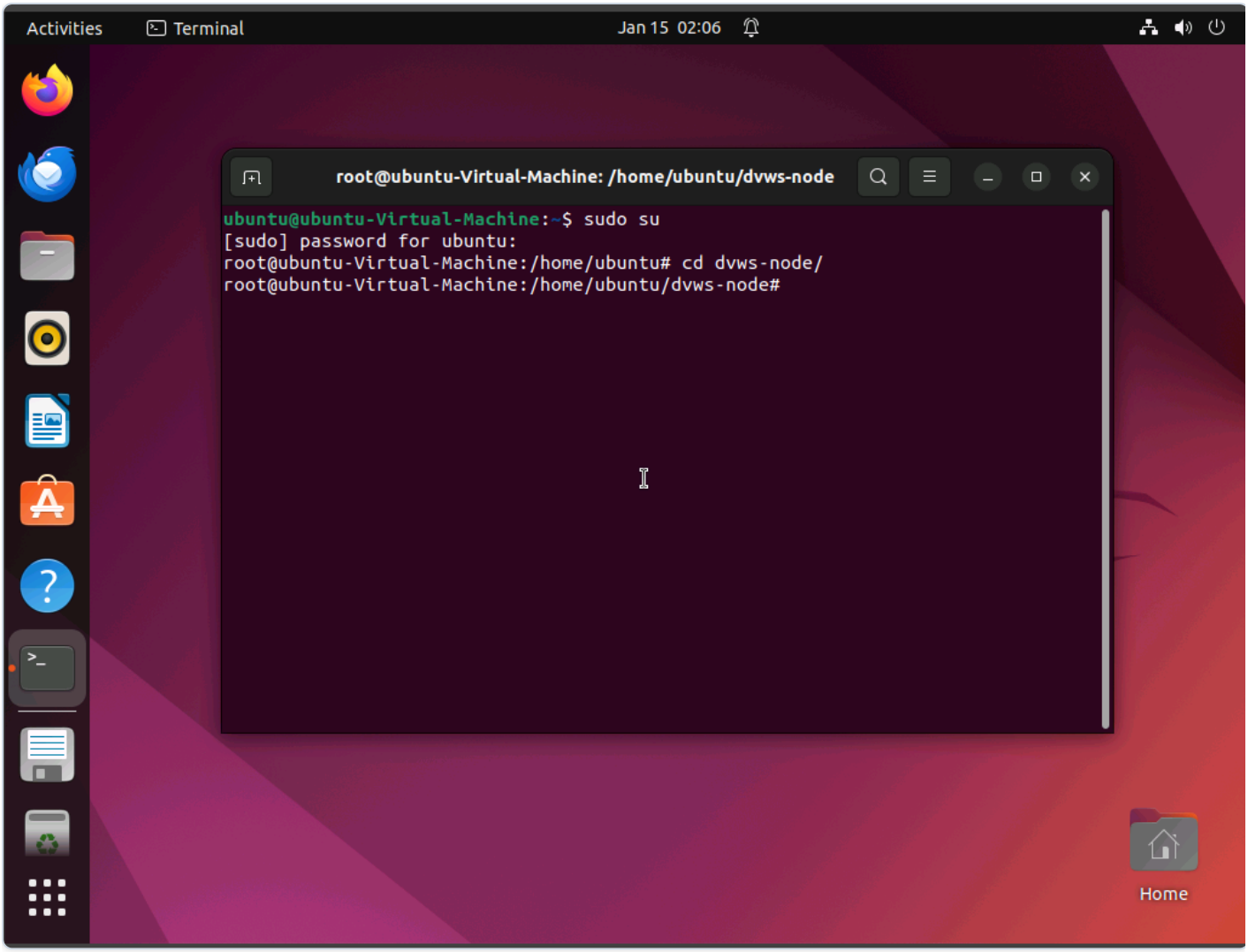
---

Task 1: Brute-Force JWT Secret Key and Forge Administrator Token

In this task, we initialize `dvws-node` on Ubuntu, log in as a standard user (`test:test`), capture the session JWT in Burp Suite, run `jwt_tool` to brute-force the HMAC key, forge admin claims (`["user:admin"]`) with key `access`, and access the Admin Area.

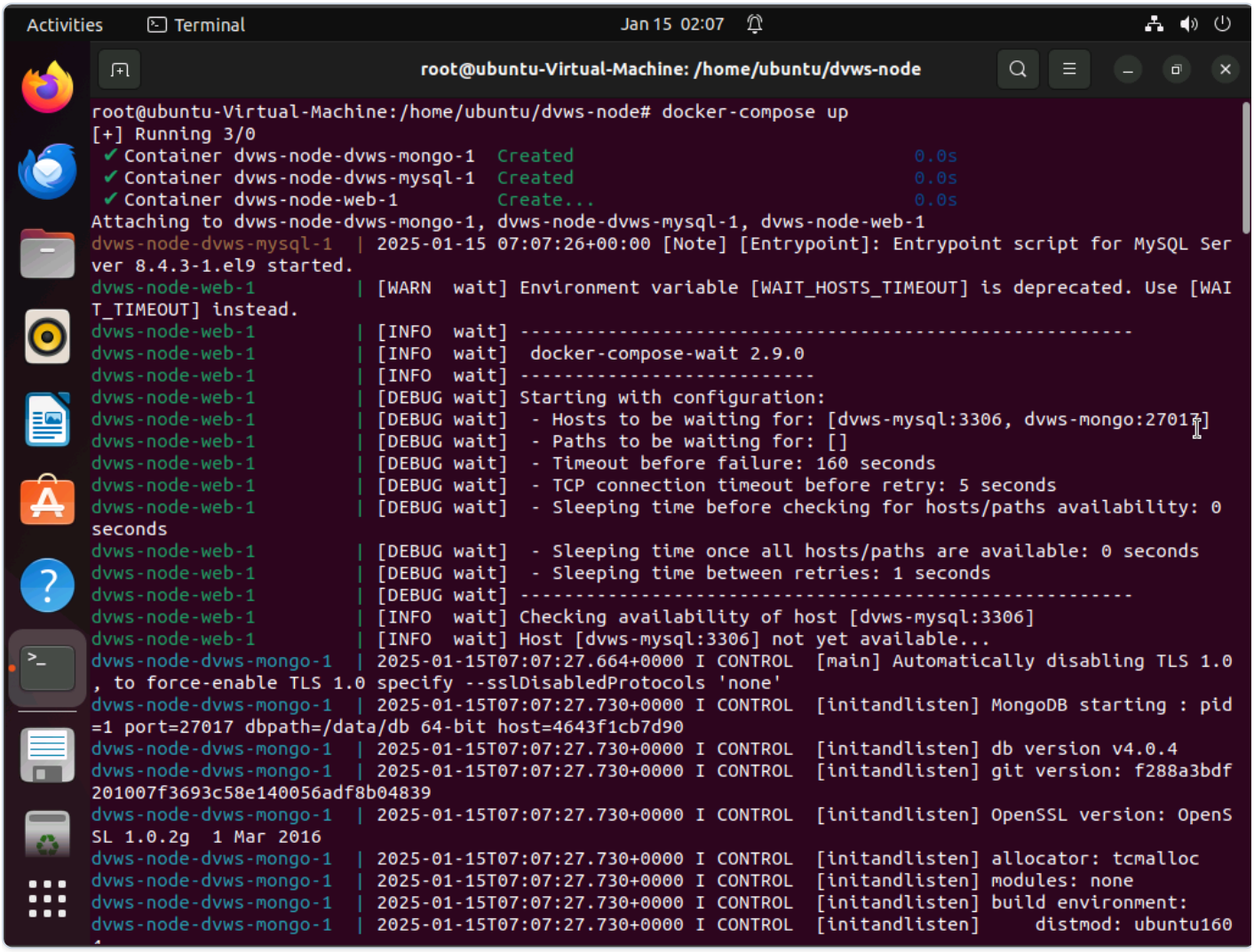
Lab Tasks

1. Switch to **Ubuntu** (**ubuntu** / **toor**). Open a terminal elevated to root (**sudo su**) and navigate to the application directory:
   cd dvws-node

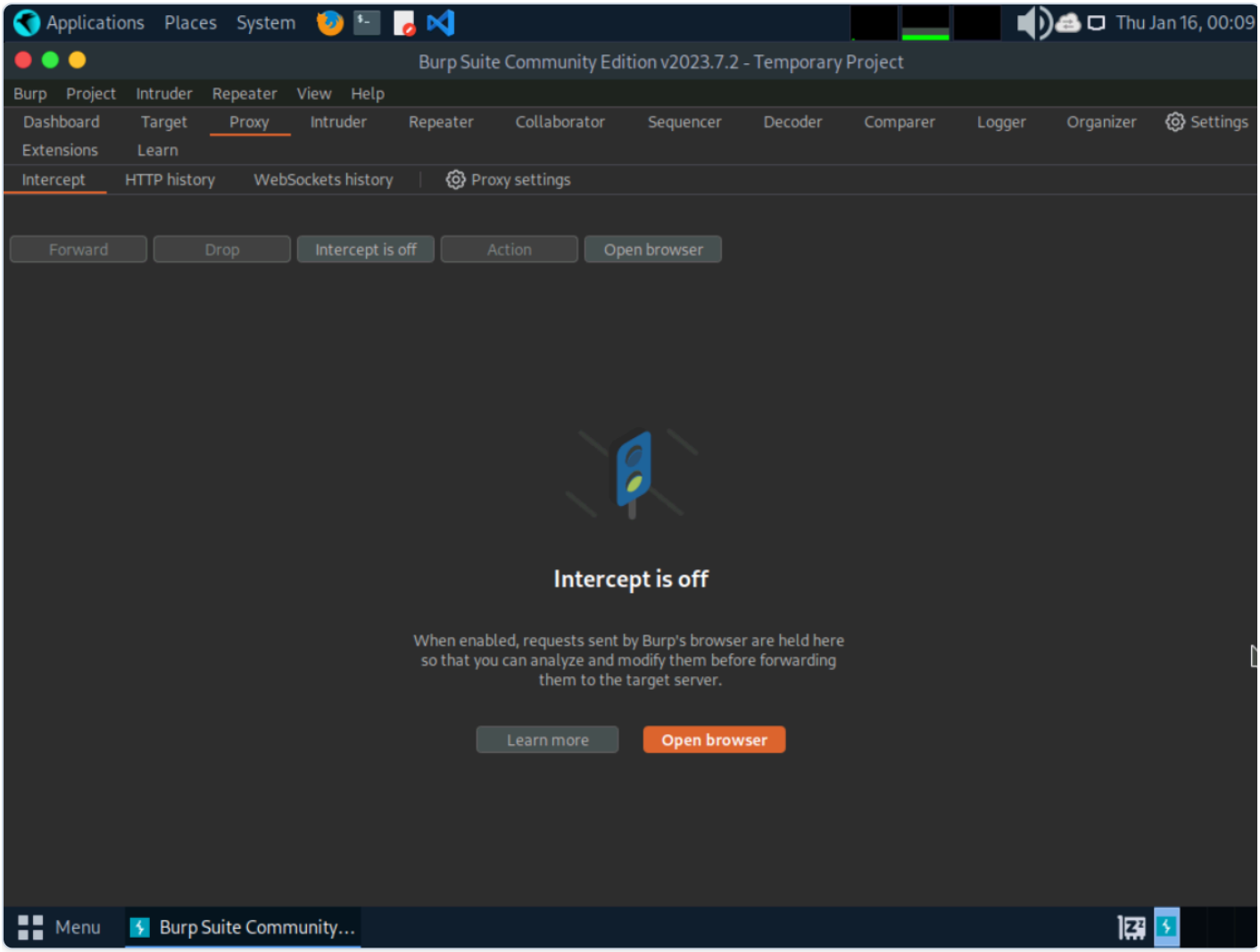


1. Start the application services:

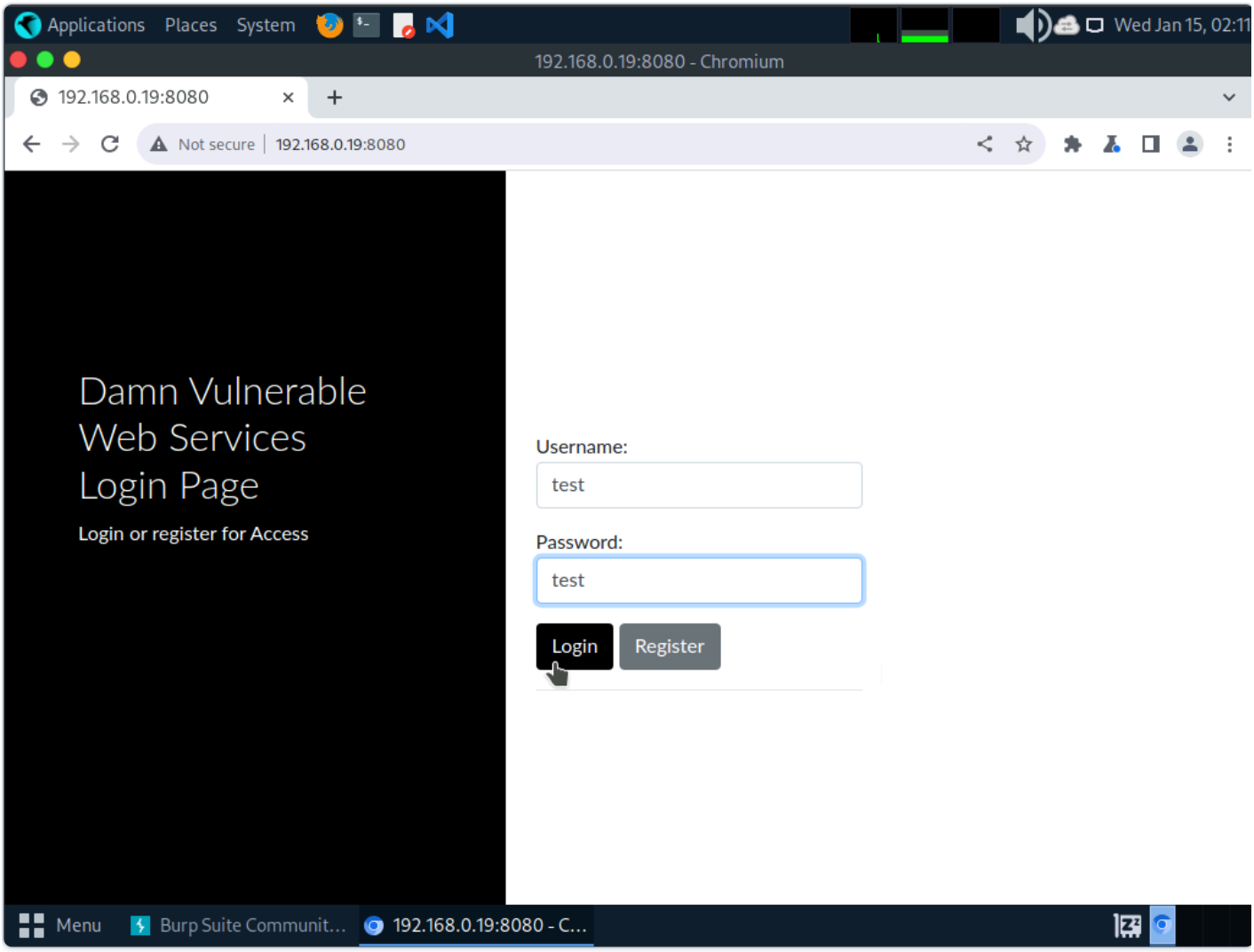
   docker-compose up



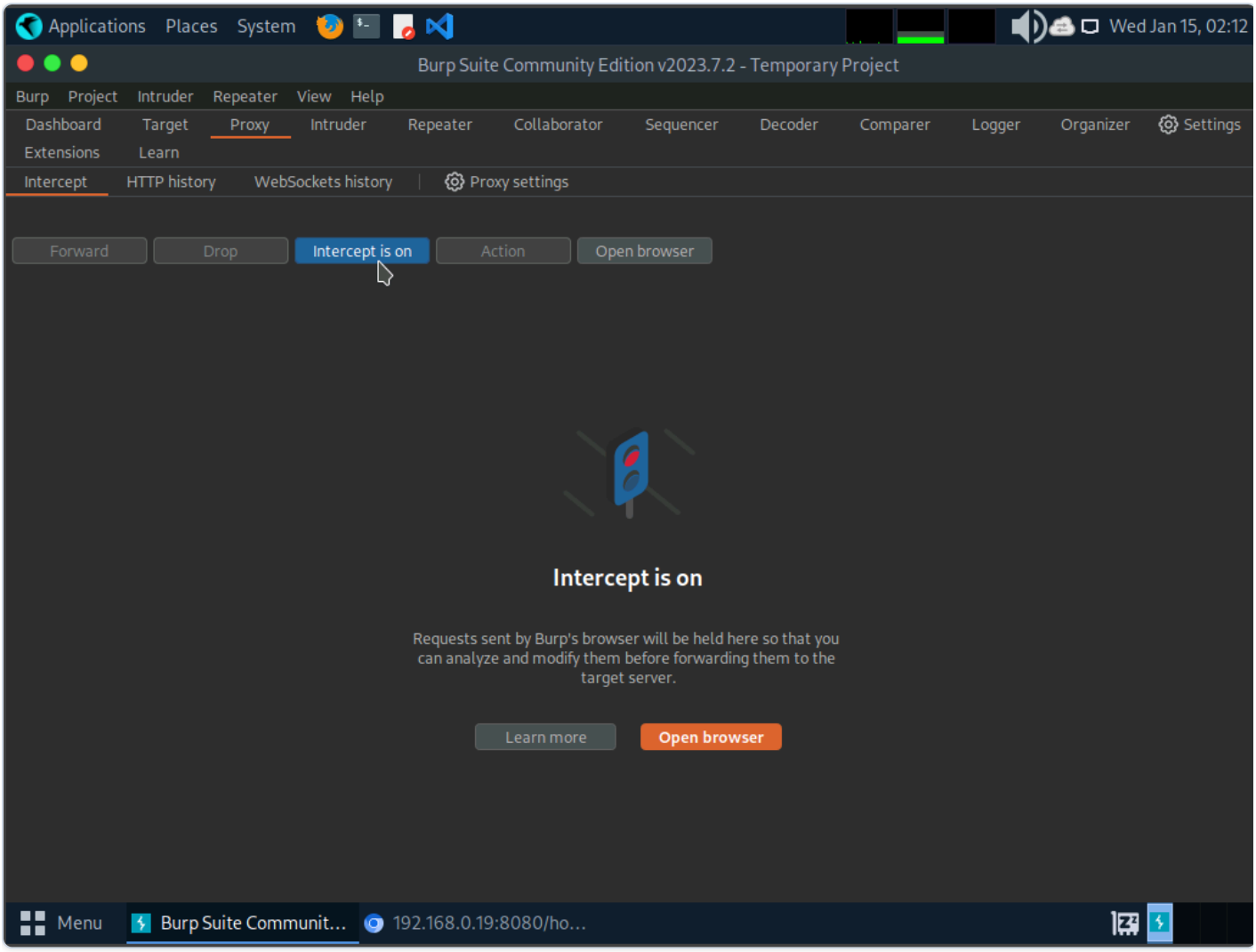
1. Switch to **Parrot Security** (**pentester** / **toor**). Launch **Burp Suite Community Edition**, select **Proxy** tab, set **Intercept is OFF**, and click **Open Browser**.



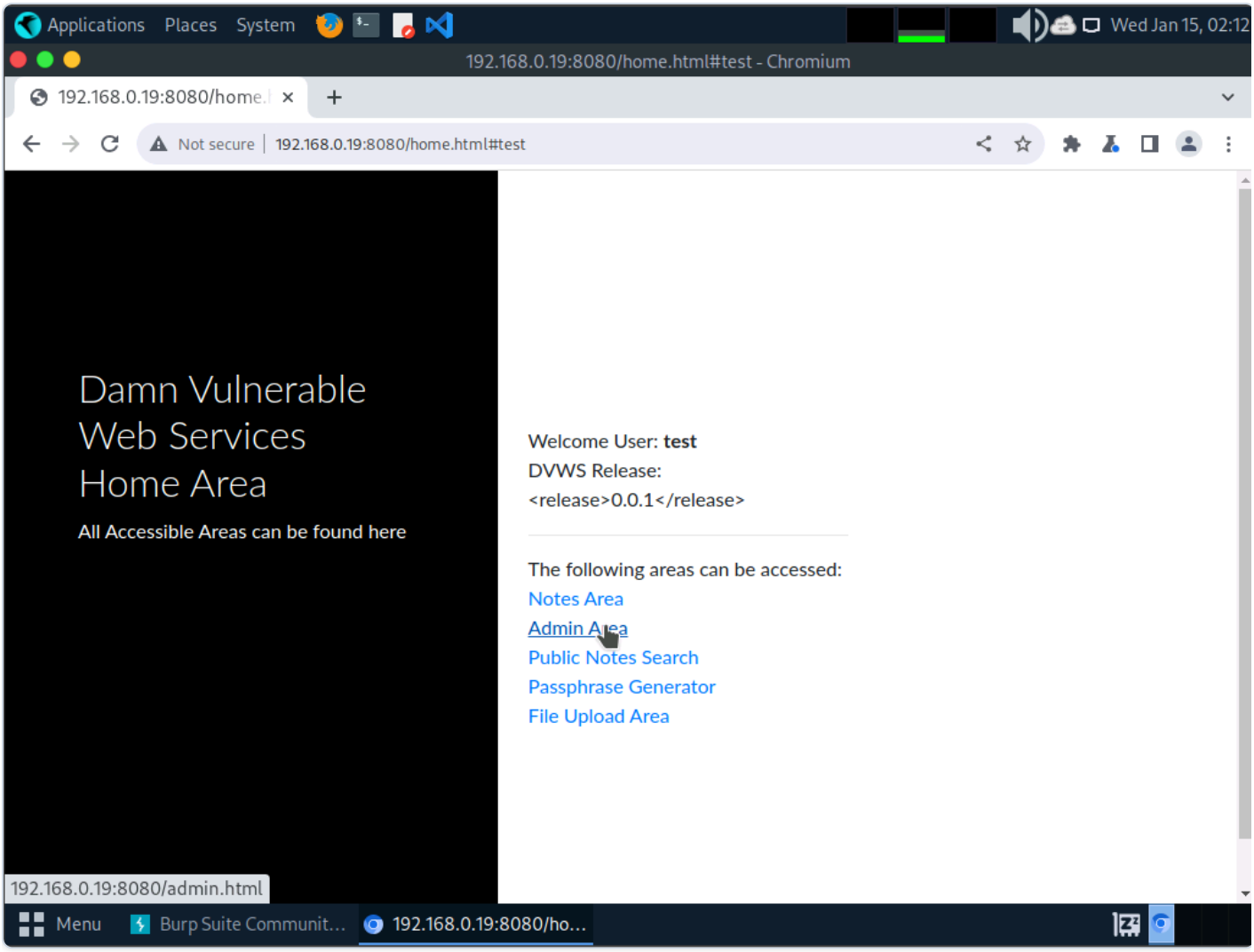
1. In Burp's browser, navigate to **http://192.168.0.19:8080** and log in with username **test** and password **test**.



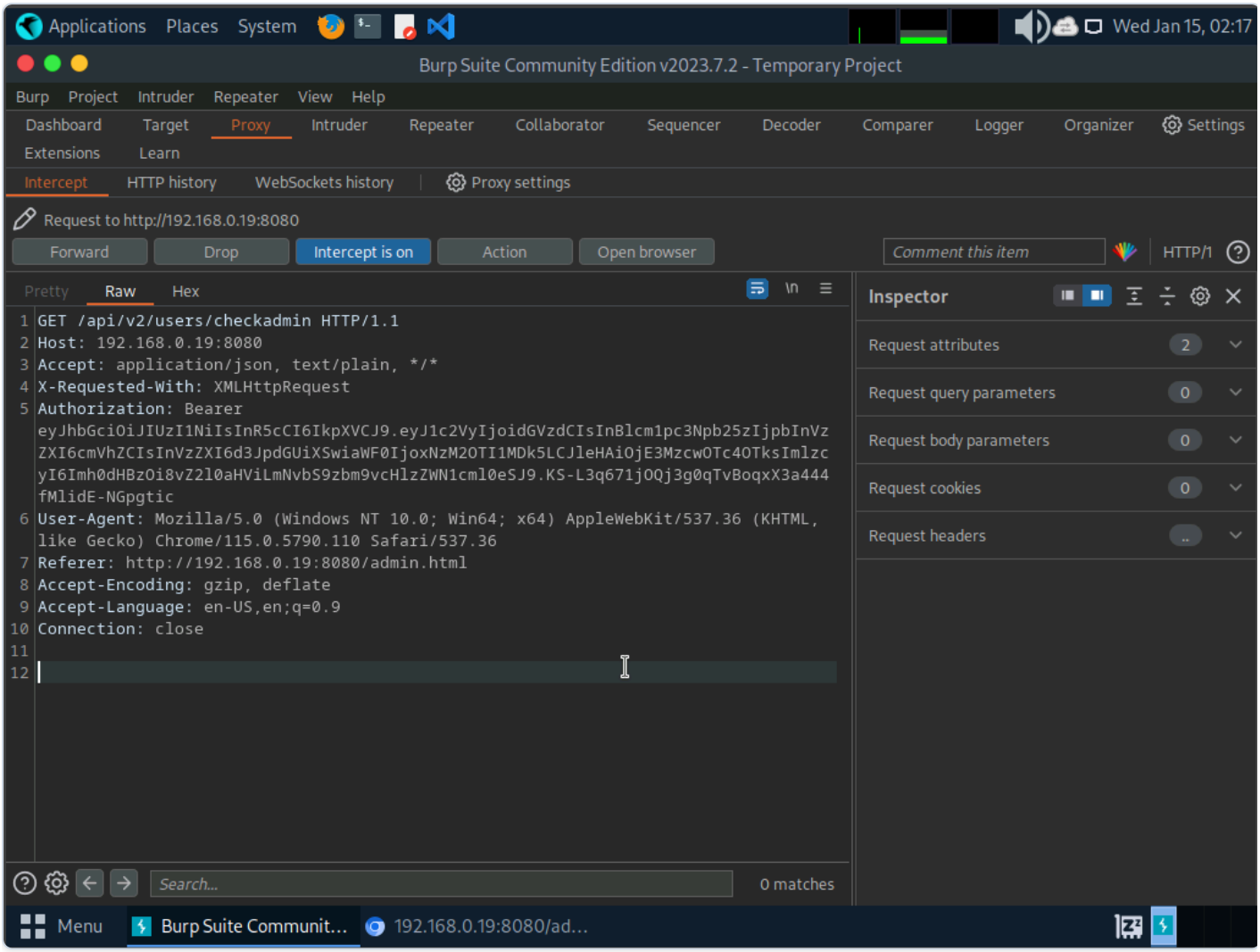
1. Return to Burp Suite and enable intercept by clicking **Intercept is OFF** to toggle to **Intercept is ON**.



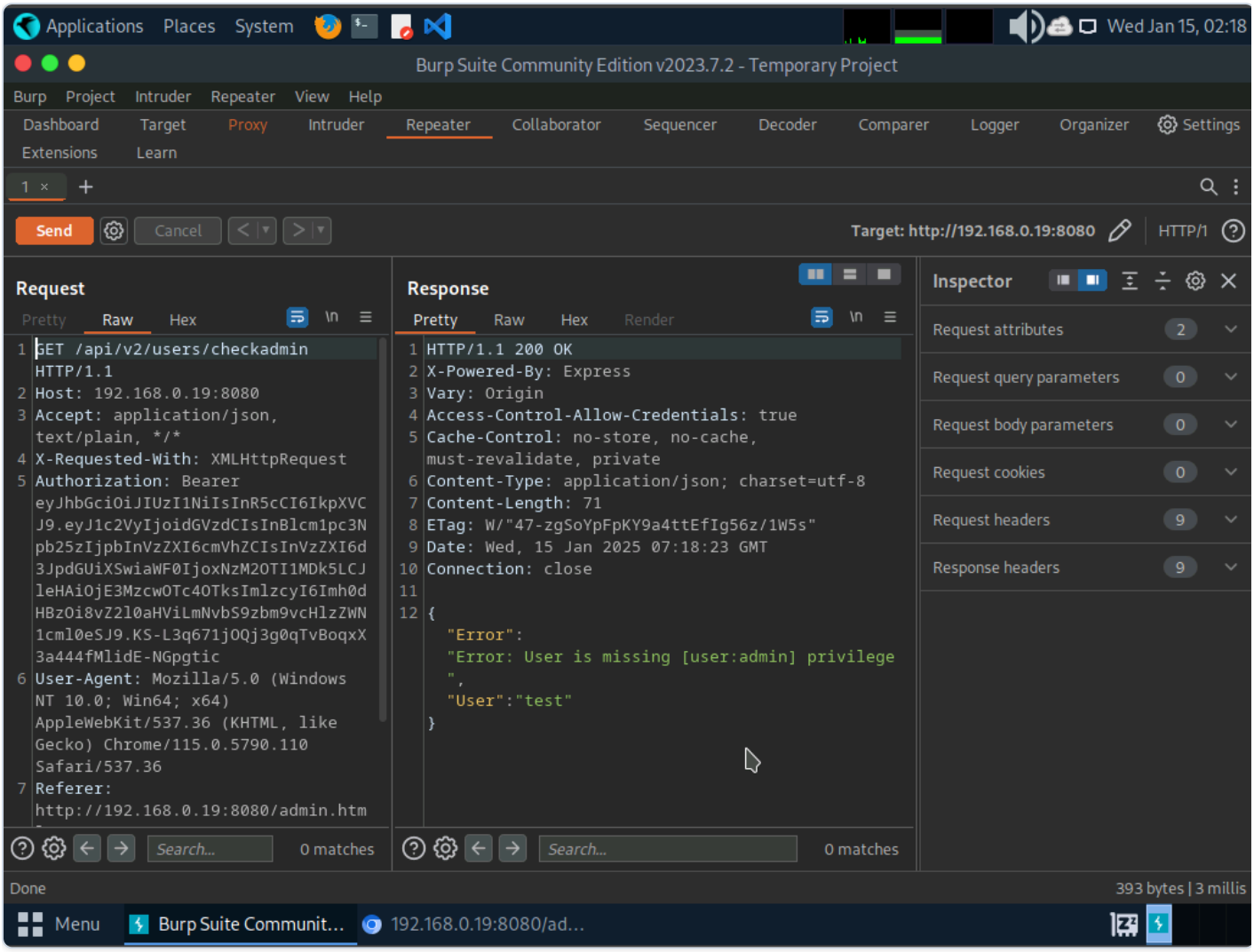
1. Switch to Burp's browser and click the **Admin Area** button.



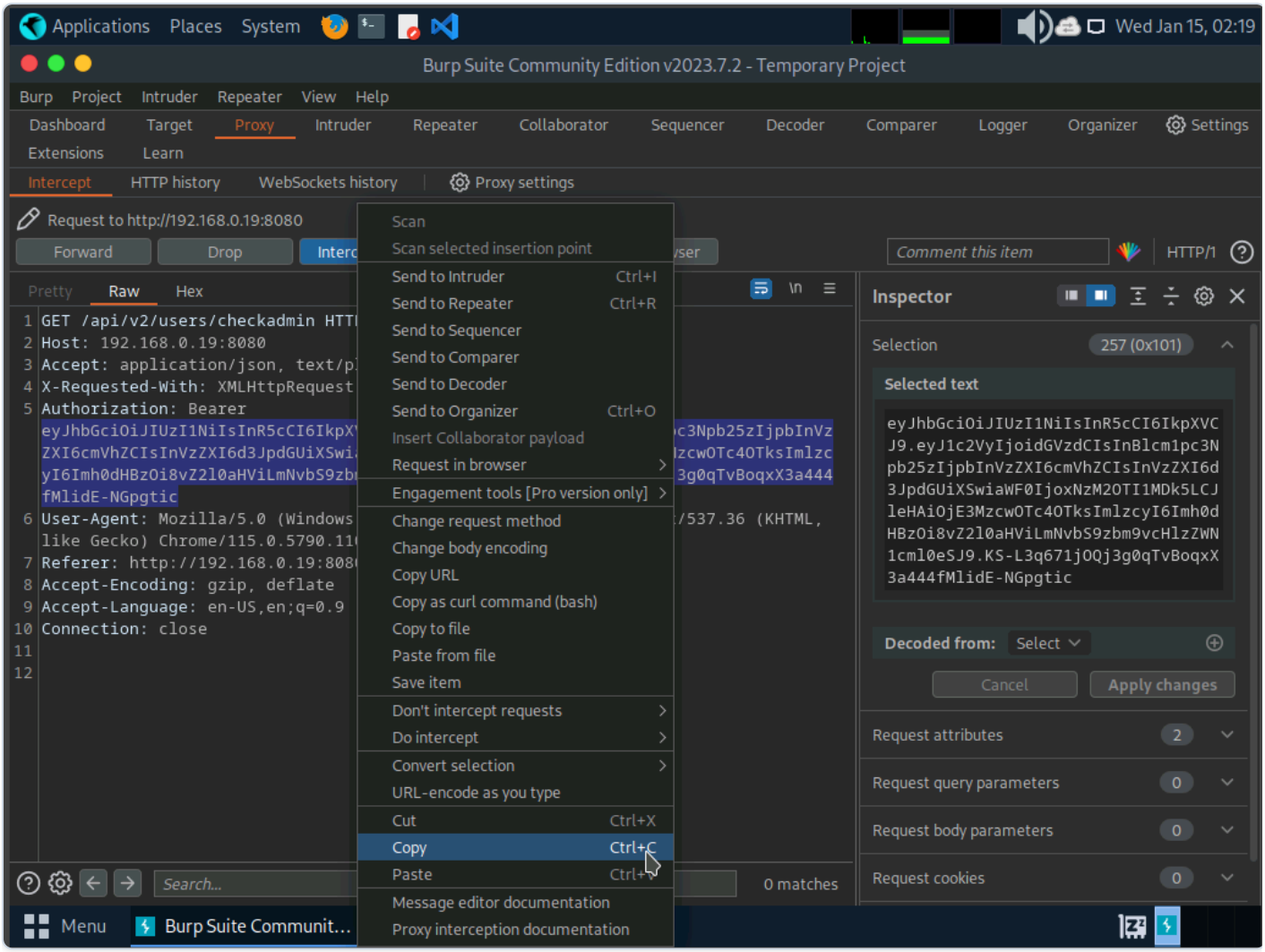
1. Switch back to Burp Suite. Click **Forward** on incoming requests until you isolate the request containing the `Bearer` token in the `Authorization:` header.



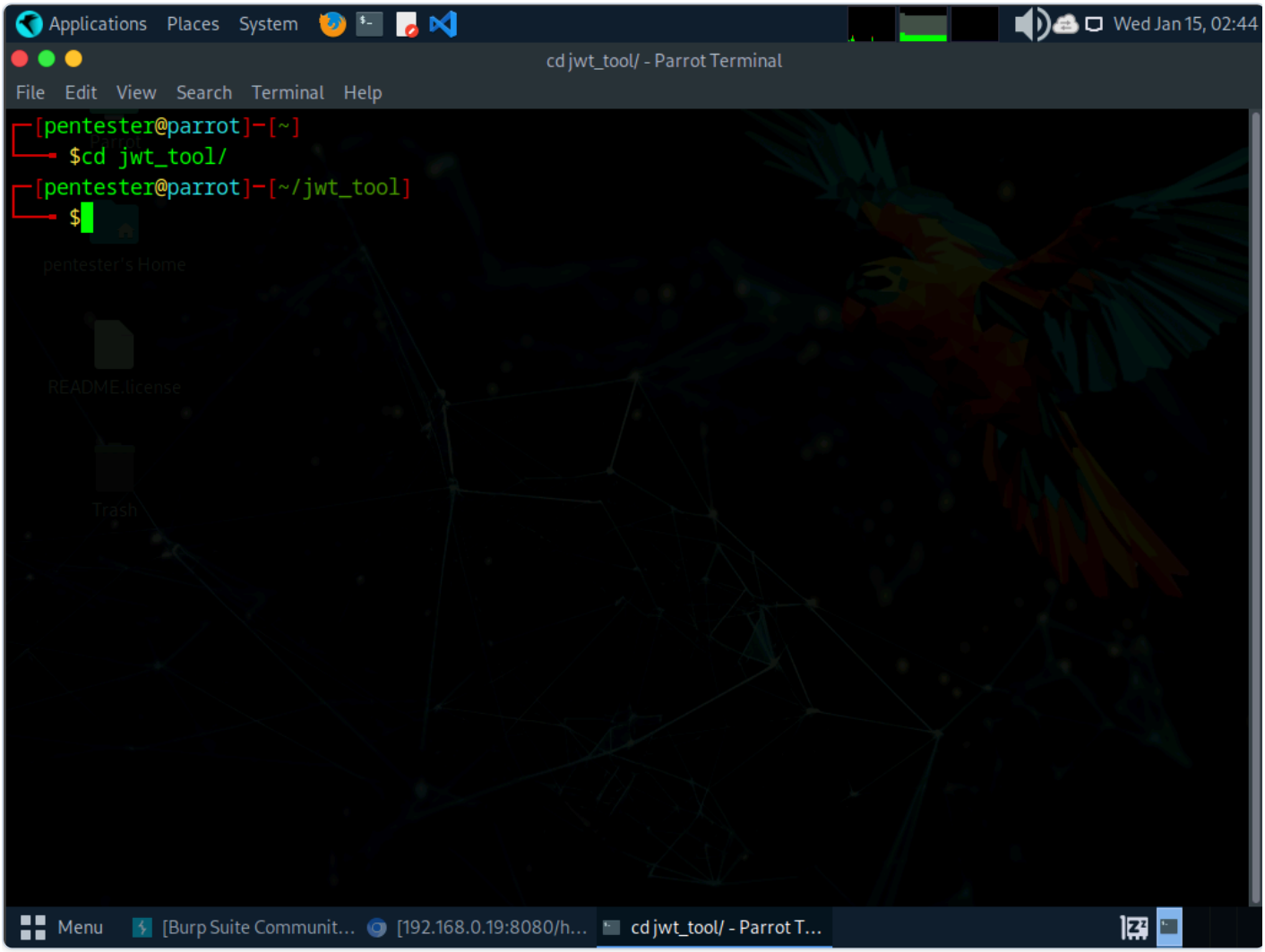
1. Right-click the request and select **Send to Repeater**.
2. Click **Send** in Repeater. Note the server error response: "*Error: User is missing [user: admin] privilege*".



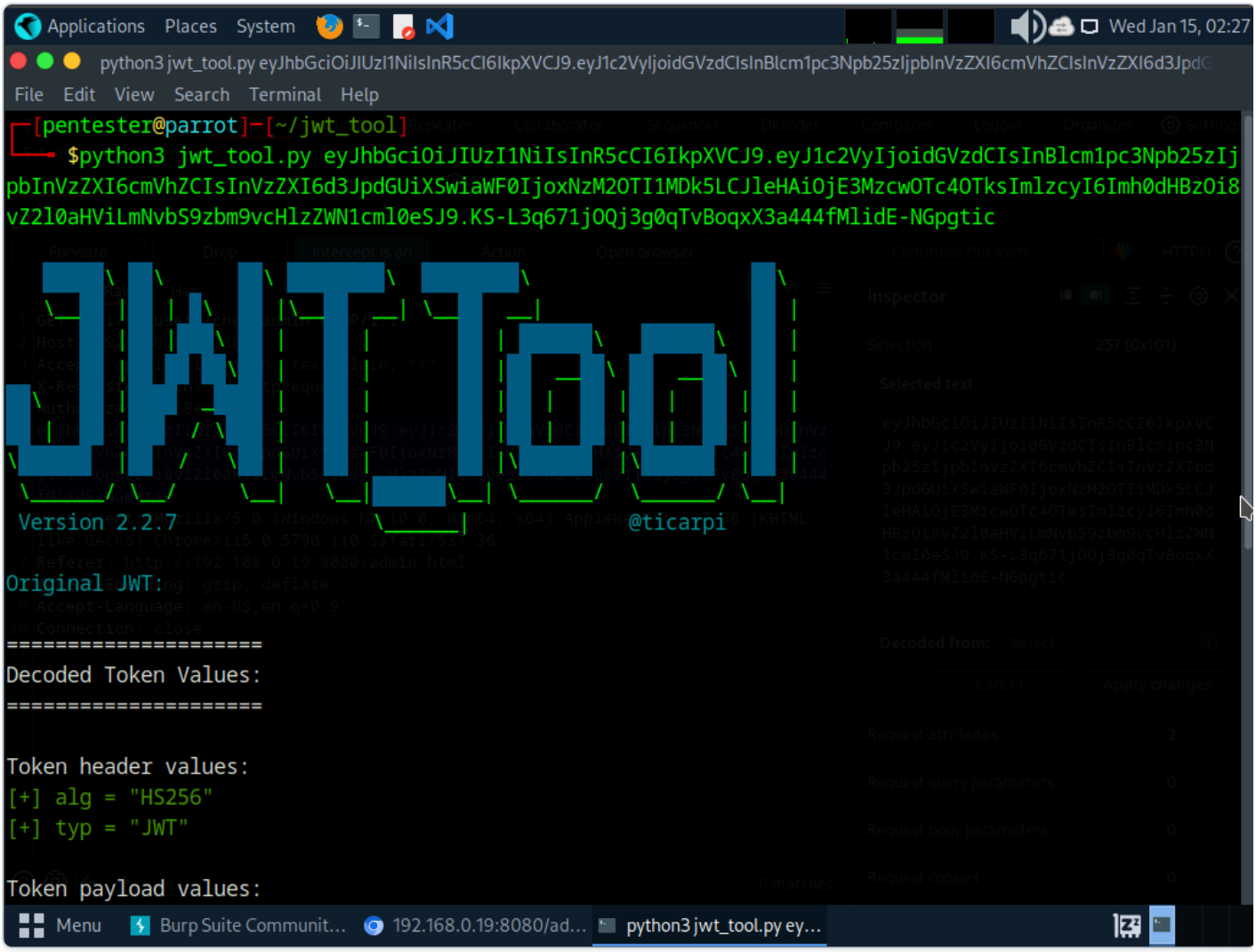
1. Return to Burp **Proxy** tab and copy the raw `Bearer` token string.



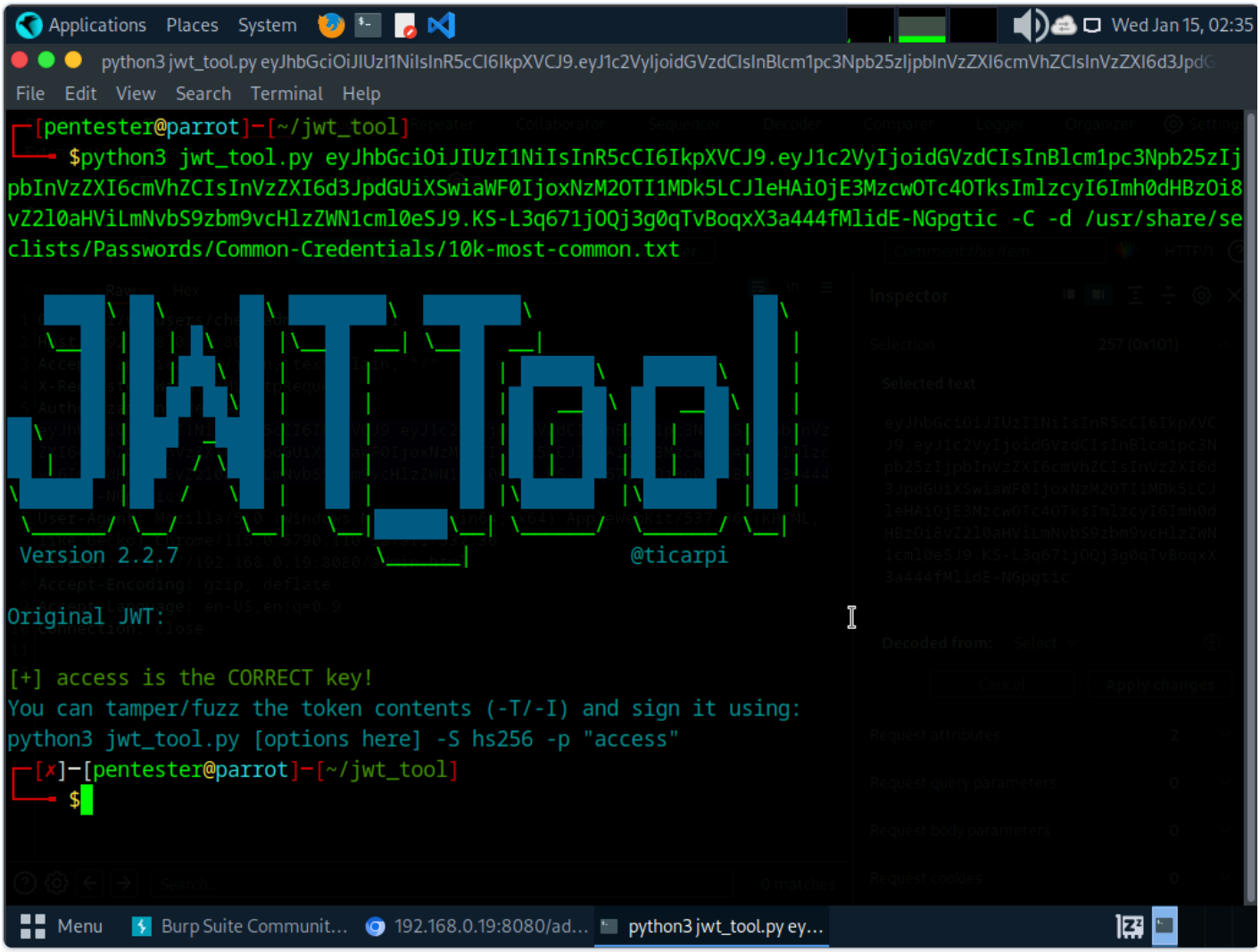
1. Open a command-line terminal on Parrot Security and navigate to the `jwt_tool` directory:
   cd jwt_tool/



1. Inspect the token header and payload structure:
   python3 jwt_tool.py <token>

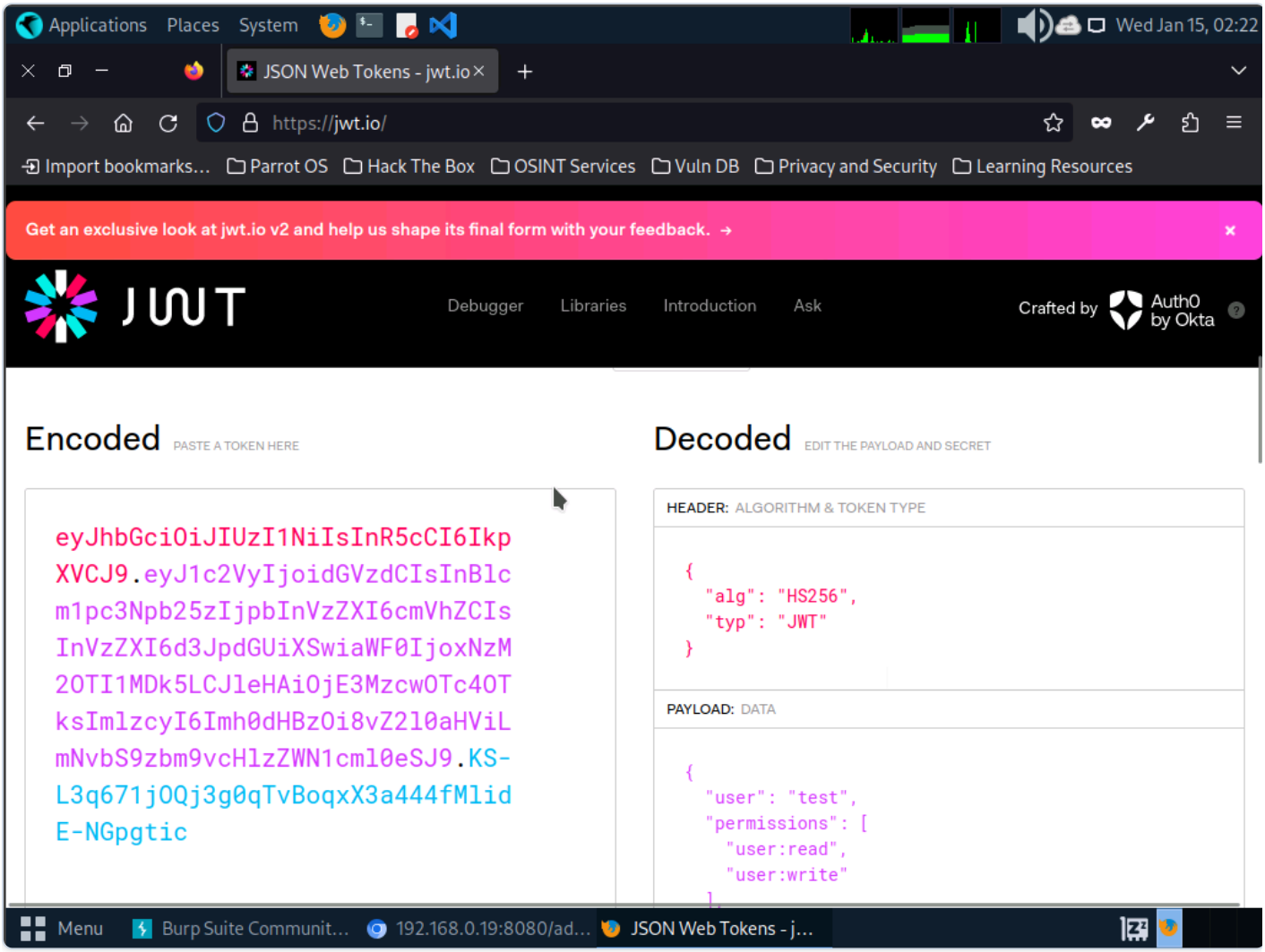


1. Execute an offline dictionary brute-force attack against the JWT HMAC key:
   python3 jwt_tool.py <token> -C -d /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt

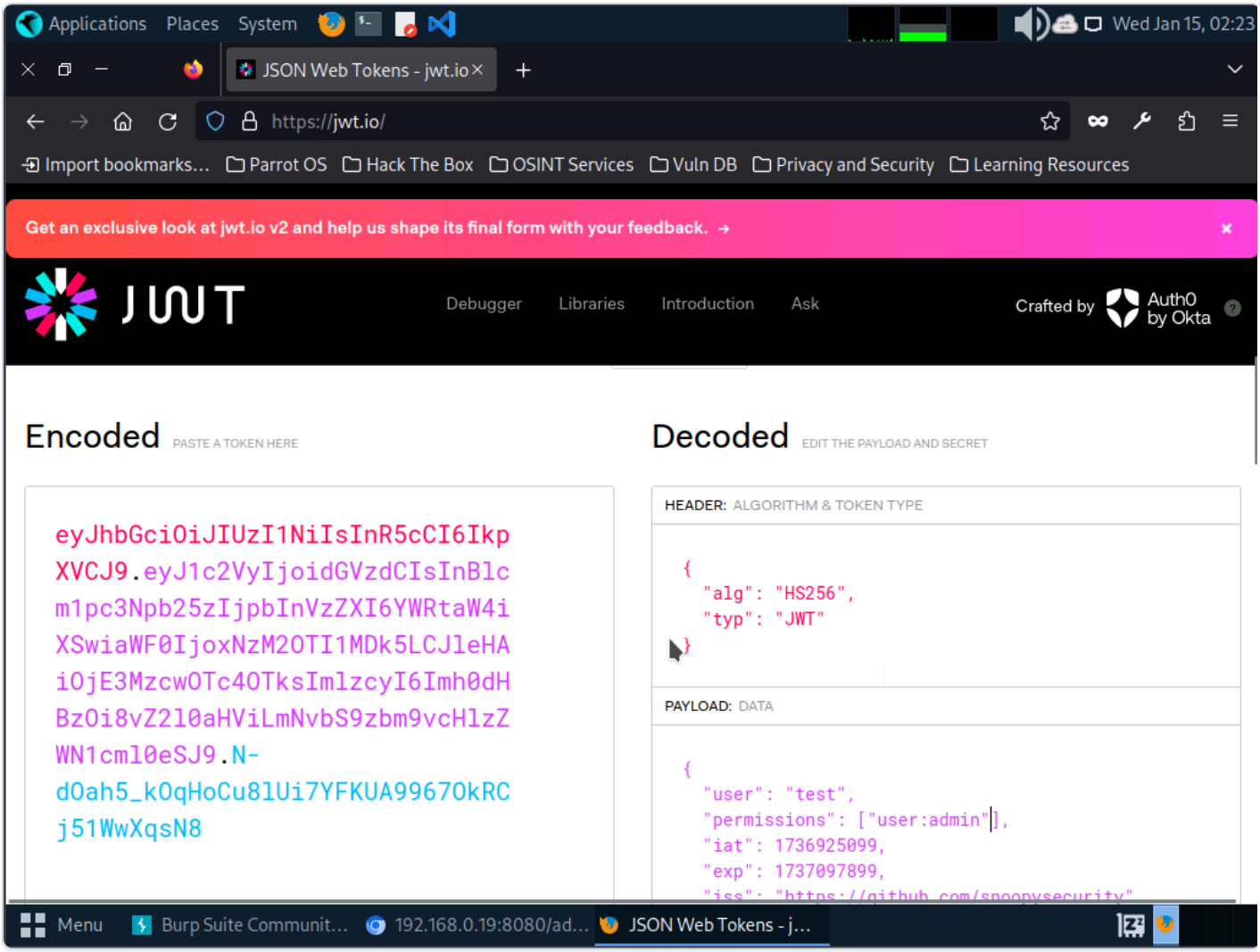


1. Observe that `jwt_tool` successfully cracking the secret signing key: **access**.

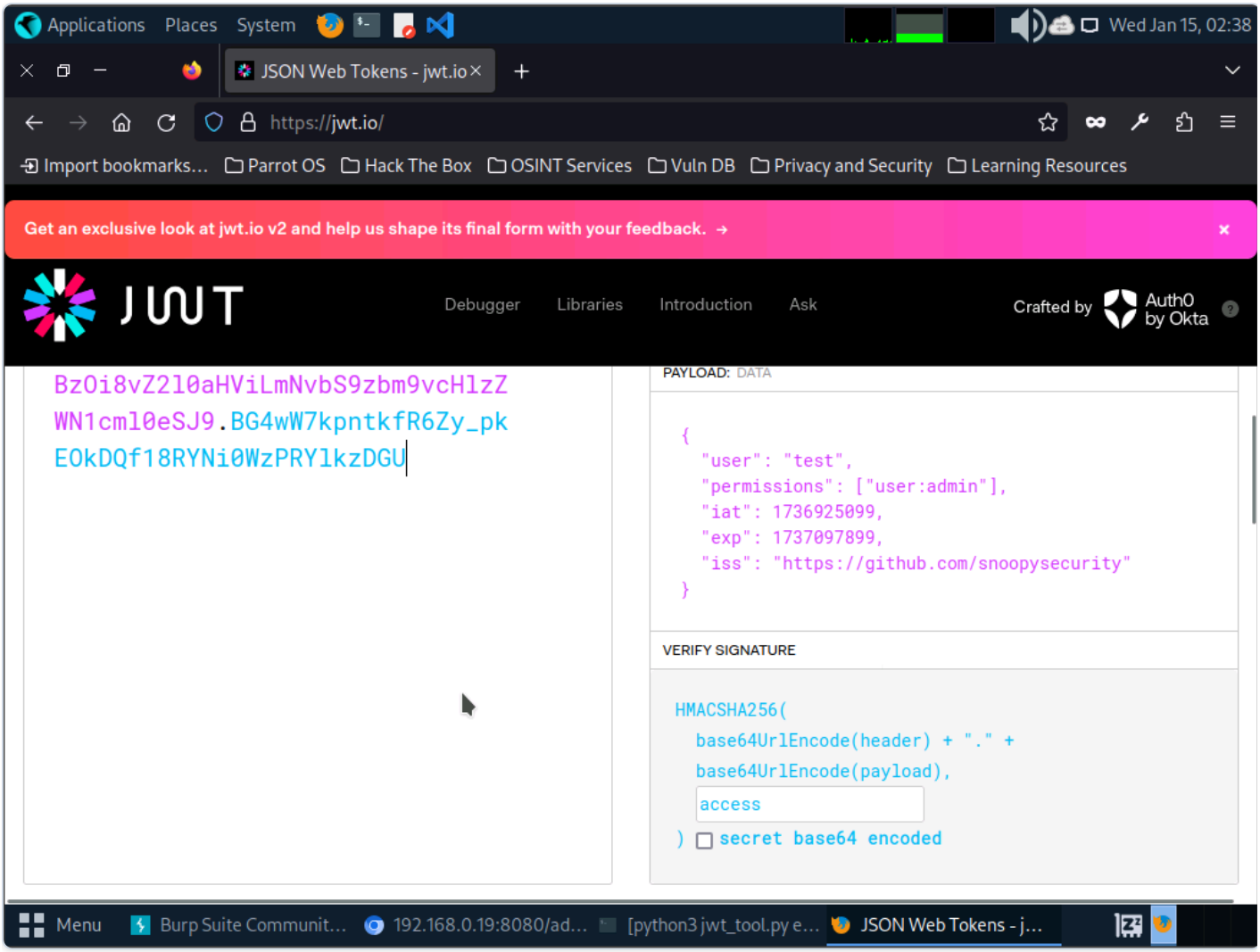
2. Open **https://jwt.io/** in a browser tab and paste the original JWT token into the **Encoded** pane.



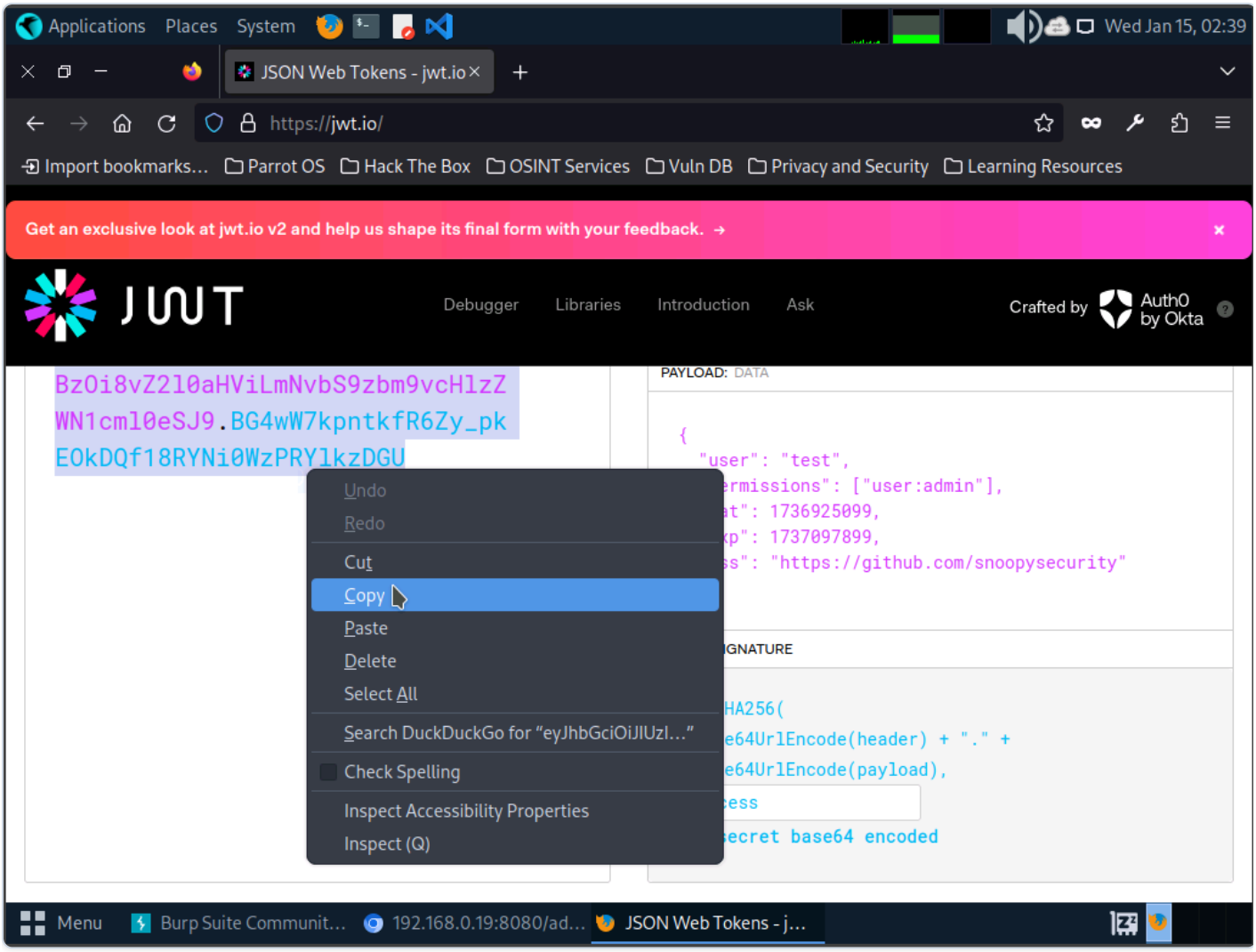
1. In the **Decoded** $
   ightarrow$ **Payload** pane, update the permissions array to grant admin rights:
   ["user:admin"]



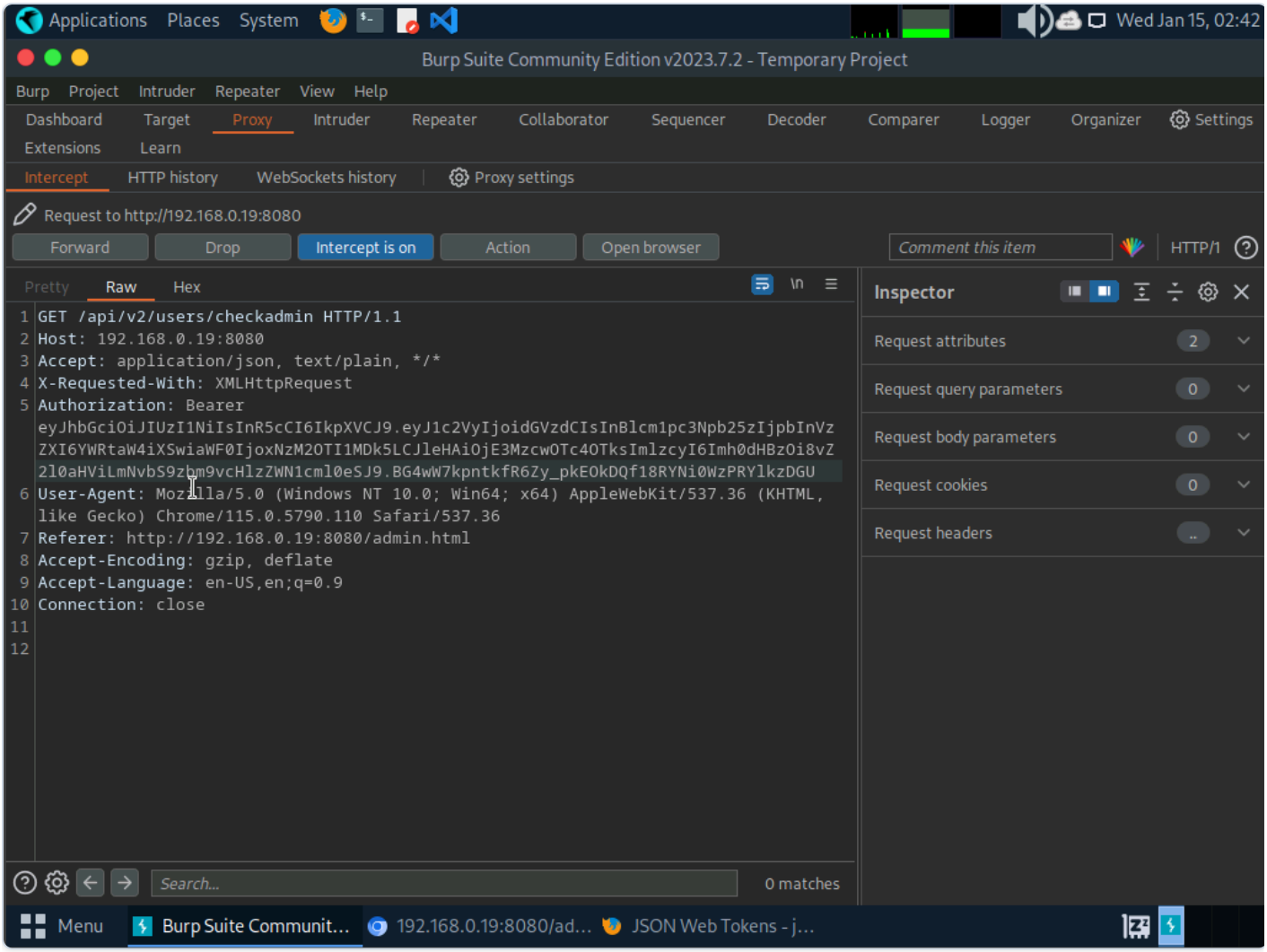
1. Under **Verify Signature**, enter the recovered secret key: **access**.



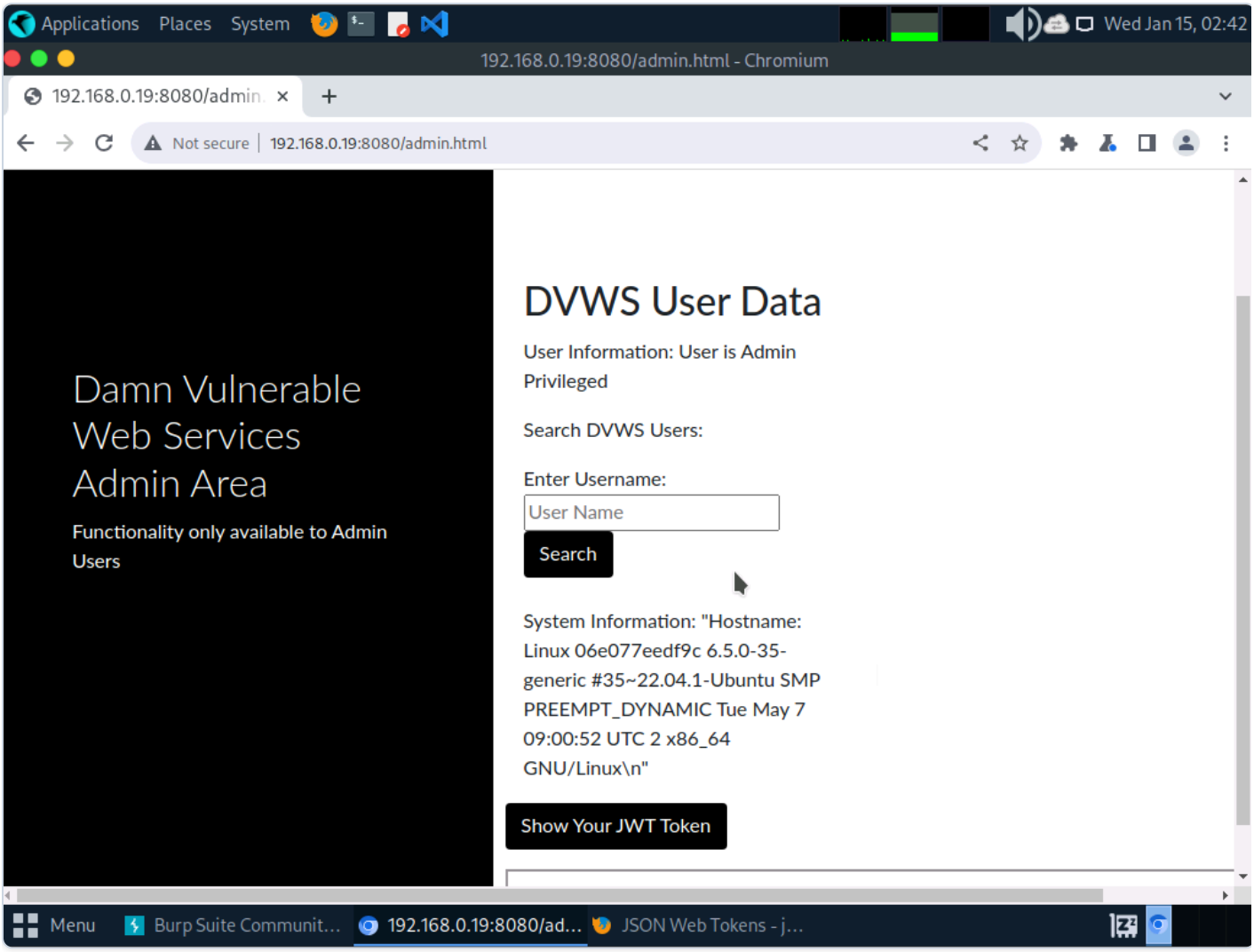
1. Copy the newly generated and signed encoded JWT token string.



1. Switch back to Burp Suite Proxy Intercept window, replace the `Authorization: Bearer <token>` value with the forged admin token, and click **Forward**.



1. Return to Burp's browser and verify that you have successfully bypassed authorization and accessed the **Admin Area** dashboard as an administrative user.



1. Close all browser and Burp Suite windows.

This concludes the lab exercise.

IMPORTANT: ❓ **Question 6.10.1.1** Using the Parrot Security machine, access DVWS and utilize automated tools to enumerate potential secret keys, analyze the JWT structure, and generate a forged token to bypass authentication or escalate privileges. Provide the secret key obtained after brute-forcing the token using jwt_tool. [!SUCCESS] ✅ **Answer:** **access**